Python Sequences, Lists, and Strings

Sequences: These apply to all sequences including lists, strings, and tuples.

General sequence tools (Sequence operators and index syntax)

Concatenation (+) - Joins two sequences to create a new sequence. Creates a deep copy.
Example: `x = [1, 2, 3] + [4, 5, 6]`
Example: `x = list1 + list2`
Example: `x = "Bob's" + "Burgers"`

Repetition (*) - Creates a new sequence that repeats the original. Creates a deep copy.
Example: `x = [1, 2, 3] * 3`
Example: `x = "b" + "an" * 2 + "a"`

The *in* reserved word (item *in* sequence) - Returns true if the *item* is in the *sequence*.
Example: `x = 3 in [1, 2, 3, 4]`
Example: `x = letter in "muffins"`
Example: `if val in myList:` (common usage)

indexing ([index or position]) - Returns one element or character from a sequence.
Example: `x = list1[3]`
Example: `c = string1[3]`

can do d['c'] for dict to get value

(k, v)

Slicing ([start : end : step]) - Returns part of the original sequence.
Example: `x = list1[2:7:2]`
Example: `x = string1[2:7:2]`

Sequence Functions:

len(sequence) - Returns the number of elements in a list or characters in a string.
Example: `x = len(list1)`
Example: `x = len(string1)`

min(sequence) - Returns the "smallest" element in a sequence.
Example: `x = min(list1)`
Example: `c = min(string1)`

max(sequence) - Returns the "largest" element in a sequence.
Example: `x = max(list1)`
Example: `c = max(string1)`

Sequence Methods: These methods must be "called on" a sequence.

index(item) - returns the index or position of the first occurrence of *item* within a sequence. The method throws a ValueError if the item is not found in the sequence.

Example:
```
list1 = [1, 2, 3]
x = list1.index(2)
```
Example:
```
string1 = "banana"
x = string1.index("an")
```

count(item) - returns the number of times *item* occurs within the sequence.

Example:
```
list1 = [1, 2, 1, 3, 1, 4]
x = list1.count(1)
```
Example:
```
string1 = "banana"
x = string1.count("an")
```

List Methods These apply only to lists, and must be "called on" a list. They modify the list, and some return a value.

append(element) - Adds an element to the end of a list.

Example: `list1.append(7)`

Example: `list1.append("apple")`

insert(index, element) - Adds an element in the specified location.

Example: `list1.insert(7, 1)`

Example: `list1.append("apple", 1)`

remove(value) - Removes the first occurrence of *value* from the list.

Example: `list1.remove(7)`

Example: `list1.remove("apple")`

pop() - Removes and returns the last element from the list.

Example: `x = list1.pop()`

pop(index) - Removes and returns the element at the given index from the list.

Example: `list1.pop(1)`

sort() - The list is now sorted from lowest to highest.

Example: `list1.sort()`

reverse() - Reverses the order of elements in the list.

Example: `list1.reverse()`

String Methods These apply only to strings, and must be "called on" a string.

isalpha() - Returns True if the string contains only letters.
Example: `x = string1.isalpha()`
Example: `if word.isalpha():` (common usage)

isdigit() - Returns True if the string contains only digits (0-9)
Example: `x = string1.isdigit()`

isalnum() - Returns True if the string contains only letters and/or digits.
Example: `x = string1.isalnum()`

islower() - Returns True if the string does not contain any uppercase letters.
Example: `x = string1.islower()`

isupper() - Returns True if the string does not contain any lowercase letters.
Example: `x = string1.isupper()`

isspace() - Returns True if the string contains nothing but whitespace (spaces, newlines, tabs)
Example: `x = string1.isspace()`

lower() - Returns a new string with all letters in lowercase.
Example: `x = string1.lower()`

upper() - Returns a new string with all letters in uppercase.
Example: `x = string1.upper()`

strip() - Returns a new string with no leading or trailing whitespace characters.
Example: `x = string1.strip()`

find(substring) - Returns the position of *substring* within a string, or -1.
Example: `x = string1.find("an")`

find(substring, start, end) - Returns the position of *substring* within a string, or -1.
Example: `x = string1.find("an", 2, 5)`

startswith(substring) - Returns True if the string starts with the *substring*.
Example: `x = string1.startswith("an")`

endswith(substring) - Returns True if the string ends with the *substring*.
Example: `x = string1.endswith("an")`

replace(old, new) - Returns a new string with all occurrences of *old* replaced by *new*.
Example: `x = string1.replace("an", "and")`

split() - Returns a list of elements separating the original string using any whitespace.
Example: `x = string1.split()`

split(delimiter) - Returns a list of elements separating the original string using the *delimiter*.
Example: `x = string1.split(";")`